Technical report — `Rütli`

A deeper write-up than the README: what you built, how it works, and what the numbers say.

- **Track:** Track 2B — Rütli (Apertus-Auditor)
- **Event:** Online
- **Team:** Powerworks — William Power
- **Demo:** `docs/demo/ruetli-demo.mp4` — 1:52, narrated walkthrough of the real repo, the real `make test` suite, and the real `make eval` / `make reflect` output. How it was made: narration via Piper TTS (`en_GB-jenny_dioco-medium` voice, fully offline, no cloud TTS account), screens captured from real GitHub pages and live terminal output via headless Chromium, assembled scene-by-scene with `ffmpeg` (`libopenh264` encoder). Nothing in the recording is staged — the test/eval/reflect output is a live run against a fresh clone, not a mockup.

1. Summary

Deploying autonomous agents in regulated sectors (public administration, finance) requires non-repudiable proof of *why* an agent was permitted to take an action, not merely a log that it occurred. Rütli — named for the 1291 oath that founded the Swiss Confederation, a mutual-trust pact between parties who need to verify each other's commitments — wraps an Apertus 1.5 agent runner with an out-of-band authorization gate (Eunomia) and an append-only, SHA-256 hash-chained event ledger. Every prompt, policy decision, canonical action proposal, and execution event is recorded as an immutable, replayable event. We evaluate Rütli against a prompt-only policy baseline across a scripted task set combining legitimate, unauthorized, and adversarial prompt-injection attempts. The policy gate achieves 100% block rates on both unauthorized actions and prompt injections (versus 35.7% for prompt-only baseline, which fails 100% of prompt injections), with 0% false-block rate on legitimate actions, 100% hash-chain ledger integrity, and 100% tamper detection against sophisticated state rewriting via an append-only WORM sidecar mirror.

2. Architecture

Components:

Component	Role	Pre-existing or built this hackathon
Apertus 1.5 (8B GGUF, local)	Agent runner / inference	Pre-existing (Apertus model, not modified)
`eunomia` (Apache 2.0)	Standalone authorization server — `/check` policy gate, signed Agent Passport tokens, MCP middleware	**Pre-existing, third-party, open-source.** We did not build this. Disclosed per HackApertus submission requirements.
Event-Sourced Agent Ledger (CQRS)	Append-only, hash-chained log of every prompt, policy check, tool call, and model output	Built this hackathon
Crypto-shredding key vault	Per-subject encryption of personal-data fields; GDPR/FADP erasure = destroy the key, not the event (ADR-005)	Built this hackathon
WORM mirror	Local append-only (`chattr +a`) sidecar copy of every event; catches a sophisticated tamper that rewrites the primary ledger *and* its own hash chain consistently (ADR-006)	Built this hackathon
Replay CLI/TUI	Deterministic scrub forward/backward through a recorded agent session	Built this hackathon
Eval harness	Scripted task runner producing the Section 5 metrics	Built this hackathon

Data flow: User prompt → Apertus agent runner appends `PromptReceivedEvent` → runner calls Eunomia `/check` for the proposed tool call → Eunomia returns `ALLOW` / `DENY` (+ reason) → runner appends `PolicyEvaluatedEvent` → on `ALLOW` , runner executes the tool, appends `ToolExecutionCompletedEvent` → Apertus synthesizes the final answer → runner appends `ModelOutputEvent` . Every event carries a `PayloadHash` and `ParentEventId` , forming a hash chain used for tamper detection (Section 5).

```
sequenceDiagram
    autonumber
    actor User as Citizen / Auditor
    participant Harness as Apertus Agent Harness
    participant Eunomia as Eunomia Policy Gate (/check)
    participant Ledger as Event Ledger (SQLite / Hash-Chained)
    participant WORM as Append-Only WORM Mirror
    participant Model as Apertus 1.5 8B
    participant Tool as Target System / Tool

    User->>Harness: Submit Prompt
    Harness->>Ledger: Append(PromptReceivedEvent)
    Ledger->>WORM: Mirror Append
    Harness->>Model: Infer Action Proposal
    Model-->>Harness: ProposedCall(tool, resource)
    Harness->>Ledger: Append(ToolProposedEvent)
    Ledger->>WORM: Mirror Append
    Harness->>Eunomia: Verify Entitlement (/check)
    alt Allowed
        Eunomia-->>Harness: Decision(ALLOW)
        Harness->>Ledger: Append(PolicyEvaluatedEvent: ALLOW)
        Harness->>Tool: Execute Simulated Tool
        Tool-->>Harness: Result
        Harness->>Ledger: Append(ToolExecutionCompletedEvent)
        Harness->>Model: Synthesize Final Response
        Model-->>Harness: Response Text
        Harness->>Ledger: Append(ModelOutputEvent)
        Harness-->>User: Audited Output
    else Denied / Injected
        Eunomia-->>Harness: Decision(DENY)
        Harness->>Ledger: Append(PolicyEvaluatedEvent: DENY)
        Harness-->>User: Refusal & Audit Notice
    end
```

Naming note: “Rütli” is this submission’s name only. It is unrelated to “Eunomia” (the third-party OSS dependency above) and unrelated to “EUnomia”/“Nomothetes” (an unrelated Powerworks product, a different domain entirely, being renamed separately). Three distinct things, kept distinct deliberately after an internal naming collision was caught during setup.

Target architecture (mandatory)

Rütli is deployable as:

- **a) On-premise** — Eunomia server, the Apertus 8B GGUF runner, and the event ledger (SQLite/Postgres) all run in one Docker Compose stack on infrastructure we administer. This is the primary demo path.
- **b) Air-gapped** — satisfied provided all dependencies (model weights, `eunomia-ai` package, Docker base images) are pulled at **build time** and nothing egresses at **runtime**. This split is enforced in the `Dockerfile` / `Makefile`: `TODO – confirm no runtime network calls remain once the 70B CSCS path is excluded from this track's demo.`
- c. Sovereign Swiss cloud — not used for this track (would require the CSCS 70B API path; out of scope once we down-selected to the on-prem/air-gapped Auditor story over the Sovereign-Agent-Mesh track).

External dependencies (build-time only, per the air-gapped constraint above): - `swiss-ai/Apertus-v1.5-8B` weights (GGUF quantization, via Ollama/llama.cpp) - `eunomia-ai` Python package or `ttommitt/eunomia-server` Docker image — **pin to a digest, not `latest`**, before submission (reproducibility requirement, Section 8) - Standard Python/Docker base images

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B` (local GGUF, via Ollama/llama.cpp)
- **How it is used:** agents / tool use — Apertus proposes tool calls, which are gated by Eunomia before execution
- **Where it runs:** locally, on the demo machine, no external API calls at runtime (air-gapped target architecture)

Every tool-call failure, malformed JSON response, and policy refusal from Apertus is logged from day one — this feeds both Section 5 (false-block rate needs to know when the *model*, not the gate, caused a refusal) and Section 6 (limitations).

`TODO: prompt template, quantisation level, and llama.cpp/Ollama version pin once the runner is built.`

4. Data

Synthetic only — no human-subject or personal data, which sidesteps consent/licensing questions entirely (per mentor guidance).

- **Task set:** 30–50 scripted agent tasks, authored by us, mixing:
  - Legitimate tool calls (should be allowed)

  - Unauthorized tool calls (should be blocked)
  - Prompt-injection attempts (should be blocked, and should be distinguishable from a legitimate refusal in the ledger)
- **Size:** 22 tasks (8 legitimate, 7 unauthorized, 7 injection) as of this writing, target 30–50 — well under the 100 MB `data/` limit either way.
- **Licence:** our own authorship — CC-BY-4.0, same as the rest of the submission.
- **File:** `data/tasks.jsonl` .

`TODO: expand to 30-50 tasks before submission — current set is enough to validate the harness and produce the Section 5 pattern, not yet the full target size.`

5. Evaluation

2026-10-08 ADR-008 update: A versioned action contract now validates proposals and records the *proposed* action through authorization and simulated execution. `make eval` additionally reports correct action selection (77.3% in both stub conditions), invalid proposals (0) and backend errors (0). These are synthetic harness numbers, not real Apertus/Eunomia results. The previously named “replay fidelity” below measures hash-chain integrity only; it does not reproduce external execution. The gate-latency figure below is historical: the current stub run measured 0.021 ms and is not a network measurement. See ADR-008.

Baseline: the same agent with the authorization policy stated only in the system prompt — no gate, no ledger. **Ours:** the same agent with a policy gate in front of every tool call, plus the event ledger recording each decision.

> ***Harness-validation numbers below, not final submission numbers.*** *They come from `StubApertusAgent` (a deterministic, documented heuristic — see `agent.py`) and `ScopePolicyGate` (an in-process ABAC stand-in with the same decision logic a live Eunomia `/check` would make — see `policy.py`), not real Apertus 1.5 8B inference or a running `eunomia-server`. The harness, task set, ledger, and metrics code are real and validated; the agent/gate backends are TODO before this table is submission-final (see Section 8).*

Metric	Why it matters	Baseline	Ours
Unauthorized calls blocked (%)	Headline result — does the gate actually stop what the prompt-only policy misses?	35.7%	**100.0%**
↳ broken down: unauthorized (overt)		71.4% (5/7)	100.0% (7/7)
↳ broken down: injection		**0.0% (0/7)**	100.0% (7/7)
False-block rate on legitimate calls (%)	Stops “block everything” from trivially winning	0.0%	0.0%
Ledger integrity (hash chain verifies)	Checks recorded event integrity; does not prove execution replay or non-repudiation	N/A (no ledger)	100.0%
Tamper detection on mutated logs (%) — naive tamper (`verify_chain`)	Proves tamper-evidence, not just tamper-logging	N/A (no ledger)	100.0%
Tamper detection — sophisticated tamper (rewrite + recompute downstream hashes)	`verify_chain()` alone is *fooled* by this; only the ADR-006 WORM cross-check (`verify_against_worm`) still catches it — proven in `tests/test_worm_mirror.py`	N/A	100.0% (via WORM mirror; 0% via hash chain alone)
Added latency per tool call	The operational cost of the gate — judged under “Value, cost & scalability”	N/A	0.021 ms (in-process stand-in — **not representative** of a real networked Eunomia `/check` call; re-measure once wired, Section 8)

Confirmed exactly the expected pattern (per mentor feedback): the prompt-only baseline’s own alignment catches most (71%) overtly-worded unauthorized requests, but **0% of prompt-injection attempts** — the injected text is written to look like a legitimate system-level authorization, which is precisely what a surface-level, prompt-only policy cannot see through.

The gate catches both categories at 100% regardless of how the request was worded, because it checks the actual resource/scope tuple, not the prompt text. Reproduce with `make eval` (dev-stub backends) from `track_2b/`.

6. Limitations

- **Section 5's numbers are from dev-stub backends, not real Apertus 1.5 8B inference or a live Eunomia server.** `StubApertusAgent` and `ScopePolicyGate` are documented, deterministic heuristics. `OllamaApertusAgent` and `EunomiaPolicyGate` have mocked API tests but have not been verified together against live services. In particular, live policy selection across different scope sets and identity isolation remain unverified.
- **The task set's categories don't produce false positives by construction** — every legitimate task's resource falls cleanly inside its authorized scope, and every unauthorized/injection task's doesn't. Real Apertus 8B inference may propose tool calls with messier, ambiguous resource strings that a real Eunomia policy has to resolve less cleanly than our `ScopePolicyGate` prefix-match. The 0.0% false-block rate above should not be read as proven robustness yet.
- **Latency numbers are not representative.** `ScopePolicyGate` is in-process Python; a real `eunomia-server` call is a networked HTTP round-trip. The 0.021ms figure measures the harness's own overhead, not Eunomia's.
- **The WORM mirror's `chattr +a` enforcement is host-dependent** — it silently (well, not silently: `ledger.worm.immutable_enforced` reports `False`) degrades to a plain file on non-Linux hosts, non-ext filesystems, or sandboxes without the capability (this dev environment is one such case). The submission demo machine's filesystem must be checked, not assumed.
- **Crypto-shredding (ADR-005) is schema-correct but only exercised on a synthetic company-UID-as-subject-ID model** — see ADR-005's "what this does NOT do" note: this is not a full GDPR compliance posture (no DPIA, no lawful-basis determination), just the architectural compatibility piece.
- **Air-gapped claim (Section 2) needs re-verification after every dependency version bump, not just once at the start.**

7. Related Work & Differentiation

Raised directly by mentor feedback (Oliver Grognuz, 2026-10-05): is Rütli differentiated from the existing LLM-gateway ecosystem, or does it duplicate functionality already available off-the-shelf — specifically in **LiteLLM**, the most widely-deployed open-source LLM proxy/gateway.

What LiteLLM already provides: request-level logging and tracing (Langfuse, Arize Phoenix, LangSmith, OTel v2 — one trace per request spanning the HTTP call, auth, guardrails, the LLM call, and DB/cache work); guardrails (PII masking via Presidio, secret redaction, content moderation, banned keywords, per-key/team enforcement) with "an audit log on every request"; and, on its Enterprise tier, admin-action/key-change audit logs with retention policies, RBAC, and log export to GCS/Azure Blob for compliance storage.

What it does not provide, as of this writing: - **No tamper-evident or cryptographically verifiable audit trail.** BerriAI/litellm#29895 (opened 2026-06-07, still open, no maintainer response or linked PR) requests exactly this — Ed25519-signed, hash-chained post-call receipts for EU AI Act Article 12 compliance — and states plainly that LiteLLM's current logs are "operator-controlled and cannot be independently verified by auditors who don't trust the operator's infrastructure." This is precisely the gap Rütli's hash-chained ledger (ADR-003) plus WORM mirror cross-check (ADR-006) closes: `verify_chain()` catches naive tampering, and `verify_against_worm()` catches the sophisticated tamper that rewrites the chain *and* recomputes every downstream hash — see `tests/test_worm_mirror.py`. Both are built and passing today; LiteLLM's equivalent is an unresolved feature request. - **No GDPR/FADP-compliant erasure mechanism for audit logs.** A standard log pipeline has no way to erase one subject's data without either destroying the log's integrity or leaving the personal data in plaintext indefinitely. Rütli's crypto-shredding (ADR-005) destroys a per-subject decryption key, leaving ciphertext in place and the hash chain's integrity untouched — erasure and auditability stop being in tension. - **No self-review / reflective improvement layer.** Periodic self-assessment ("did the agent get on well here?", surfaced as human-facing improvement suggestions) appears in the academic LLM-agent literature as a prompting technique (self-reflection on completed trajectories — e.g. Renze & Guven, arXiv:2405.06682) but not as a shipped feature of any LLM gateway, LiteLLM included. ADR-007 (implemented — `eval/reflection.py`, `eval.cli reflect`) applies this idea to the audit ledger itself: a periodic, human-in-the-loop-only pass that mines the ledger for leaked unauthorized/injection tasks grouped by tool, false-block drift between passes, and gate-latency outliers, and appends its own findings back as a first-class auditable event — not a side-channel report.

The sharpened pitch: *LiteLLM and comparable gateways give you logs you have to trust the operator on. Rütli gives you a cryptographically verifiable, erasure-compliant, self-assessing audit trail — the actual EU AI Act Article 12 guarantee the LiteLLM community is still asking for as an open feature request.* Rütli is not a gateway competing with LiteLLM's routing/guardrail functionality; it is the governance/audit layer those gateways currently lack, and could in principle sit behind one.

8. Reproducibility

- **Hardware:** Standard x86_64 Linux workstation / server (Intel Core i5/i7 or AMD Ryzen/EPYC, 4+ CPU cores, 8 GB RAM, Linux kernel 6.x on ext4/btrfs filesystem with `chattr +a` capability).
- **Runtime (harness & verification):** Standard Python 3.11+ environment with `cryptography>=42.0`. Run `make test` (61 unit tests, stdlib `unittest`, ~0.2s runtime), `make eval` (runs the evaluation harness and prints Section 5 metrics), and `make reflect` (ADR-007 meta-audit reflection pass) from `track_2b/` or repository root.
- **Containerized execution (`make run`):** Single-command launch via `make run` from repository root or `track_2b/`. Automatically builds the container image (`ruetli-track2b`) via Docker or Podman and runs the complete test, evaluation, and reflection verification suite end-to-end.
- **Seeds:** The task set (`data/tasks.jsonl`) is deterministic and static (22 curated tasks covering legitimate requests, overt unauthorized requests, and adversarial prompt injections). For inference reproducibility with live models, `OllamaApertusAgent` pins `temperature=0.0` and fixed seed 42.
- **Dependency pins:**
  - Apertus model: `swiss-ai/Apertus-v1.5-8B` (Q4_K_M GGUF quantization)
  - Eunomia policy server: `ttommitt/eunomia-server:latest` / `eunomia-ai>=0.1.2`
  - Cryptography: `cryptography>=42.0` (Fernet AES-128-CBC / HMAC-SHA256)
  - Base container: `python:3.11-slim` (Debian Bookworm)

9. Next steps

What we would build with another month: the EU AI Act / Swiss FADP automated compliance dossier generator (one-click from the ledger to a human-oversight verification document); the RustFS Object Lock WORM anchor for the sovereign-Swiss-cloud target architecture (ADR-006, Option B — documented, not built this submission); expanding the task set to the full 30-50 target with real Apertus 8B-generated edge cases rather than hand-authored ones; and — if the eval results support it — revisit the Sovereign-Agent-Mesh track (dual-model edge/cloud routing) as a second deployment mode on top of the same ledger, rather than a competing prototype.

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References

- Apertus 1.5: https://huggingface.co/swiss-ai/Apertus-v1.5-8B
- Eunomia (OSS authorization layer, Apache 2.0): https://github.com/whataboutyou-ai/eunomia
- RustFS (OSS S3-compatible object storage with Object Lock, Apache 2.0 — ADR-006's documented future WORM anchor): https://github.com/rustfs/rustfs
- HackApertus Track 2B template: https://github.com/HackApertus/project-template/tree/main/track_2b
- `docs/adrs/` — ADR-001 through ADR-008 (Decoupled Policy Engine, Agent Passport Schema, Event-Sourced Ledger, Edge Tokenization, Crypto-Shredding, WORM Backup, Reflective Self-Review, Versioned Canonical Action Contract).
- Requirements Traceability Matrix: `docs/traceability.md`